

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по практической работе №1
по дисциплине «Обучение с подкреплением»
Тема: Сравнительный анализ алгоритма
Broad Critic Deep Actor (BCDA) с алгоритмом PPO
в среде Pendulum-v1

Студенты гр. 1304

Преподаватель

Татанов С.М.

Вотяков Ф.А.,

Глазунов С.А.

Санкт-Петербург

2026

Цель работы

Целью работы является сравнительный анализ современного гибридного алгоритма обучения с подкреплением Broad Critic Deep Actor (BCDA), предложенного в статье S. Thalagala, P. K. Wong, X. Wang «Broad Critic Deep Actor Reinforcement Learning for Continuous Control» (arXiv:2411.15806, 2024), с классическим алгоритмом Proximal Policy Optimization (PPO), изученным в рамках практической работы №2. Сравнение проводится на эталонной среде непрерывного управления Pendulum-v1.

Задание

В ходе выполнения работы необходимо:

- изучить идею гибридной архитектуры BCDA, в которой критик реализован как широкая обучающая система (Broad Learning System, BLS), а актер — как глубокая нейронная сеть (DNN);
- реализовать алгоритм BCDA для среды непрерывного управления Pendulum-v1;
- использовать в качестве базового алгоритма для сравнения реализацию PPO из практической работы №2;
- провести обучение обоих алгоритмов в идентичных условиях (одинаковая среда, число эпизодов, набор случайных начальных значений — сидов);
- сравнить алгоритмы по скорости обучения, итоговому качеству политики и устойчивости результата к выбору сида.

Сравниваемые алгоритмы

Принципиальное различие между алгоритмами заключается в способе построения и обучения критика — функции оценки ценности. В классическом PPO критик является глубокой нейронной сетью и обучается методом обратного распространения ошибки (градиентный

спуск); алгоритм работает в режиме on-policy и использует стохастическую гауссову политику. В алгоритме BCDA критик реализуется как широкая обучающая система (BLS): входные проекции признаков задаются случайно и остаются фиксированными, а выходные веса вычисляются аналитически методом гребневой (ridge) регрессии. Это устраняет необходимость в итеративном градиентном обучении критика и, согласно статье, снижает требования к объёму данных и вычислительным затратам. Актор в BCDA — детерминированная DNN, обучаемая через детерминированный градиент политики (по аналогии с DDPG/TD3), а сам алгоритм работает off-policy с буфером воспроизведения.

Алгоритм BCDA (статья). Критик-BLS получает на вход вектор «состояние + действие». Первый слой формирует узлы признаков $Z = \tanh(x \cdot W_z + b_z)$, второй — узлы расширения $H = \tanh(Z \cdot W_h + b_h)$. Матрицы W_z , W_h инициализируются случайно и не обучаются. Признаковое описание $\phi = [Z, H]$ линейно отображается в оценку ценности $Q = \phi \cdot W_{out}$. Выходные веса находятся аналитически: $W_{out} = (A^T A + \lambda I)^{-1} A^T y$, где A — матрица признаков мини-батча, y — целевые значения ценности (TD-цель), λ — коэффициент регуляризации. Для обновления актора используется аналитический градиент $\partial Q / \partial a$, вычисляемый дифференцированием BLS по действию.

Алгоритм PPO (практическая работа №2). Политика параметризуется сетью PolicyNet (MLP с активацией Tanh) и задаёт гауссово распределение действий $\text{Normal}(\mu, \sigma)$ с обучаемым логарифмом стандартного отклонения. Критик ValueNet оценивает ценность состояния. После сбора траектории фиксированной длины вычисляются преимущества методом GAE, после чего политика обновляется через clipped surrogate objective с ограничением шага

коэффициентом clip. Критик минимизирует MSE между предсказанной ценностью и целевой доходностью. Применяются нормализация преимуществ и градиентный клиппинг.

Методика эксперимента

Обучение проводилось в среде Pendulum-v1 библиотеки Gymnasium. Среда описывает маятник, который необходимо привести в верхнее вертикальное положение и удерживать там; пространство наблюдений трёхмерно ($\cos \theta$, $\sin \theta$, угловая скорость), управляющее воздействие — непрерывный момент силы. Награда на каждом шаге отрицательна и штрафует отклонение от вертикали, угловую скорость и величину момента, поэтому суммарная награда за эпизод всегда отрицательна, а её рост (приближение к нулю) соответствует улучшению политики.

Для корректности сравнения оба алгоритма обучались в идентичных условиях: по 250 эпизодов на каждый запуск, с тремя случайными начальными значениями (сидами) [7, 17, 42]. На графиках приводится усреднённая по сидам кривая с полосой $\pm\sigma$. Пунктирная линия на уровне -200 служит ориентиром «хорошо обученной» политики для данной среды.

Ключевые гиперпараметры BCDA: число узлов признаков $n_z = 60$, число узлов расширения $n_h = 120$, коэффициент регуляризации $\lambda = 10^{-4}$, скорость обучения актора 10^{-3} , $\gamma = 0.99$, мягкое обновление целевых сетей $\tau = 0.005$, размер мини-батча 128, ёмкость буфера 100 000, стандартное отклонение шума исследования 0.2, отложенное обновление критика (раз в 4 шага) и актора (раз в 2 шага). Гиперпараметры PPO: $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, $\text{lr_policy} = 3 \times 10^{-4}$, $\text{lr_value} = 10^{-3}$, длина траектории 2048 шагов, 10 эпох обновления, размер мини-батча 64, clip = 0.2, градиентный клиппинг 0.5.

Реализация критика BLS

Ниже приведён ключевой фрагмент реализации — класс широкой обучающей системы (BLS), выступающей критиком в алгоритме BCDA. В отличие от нейросетевого критика PPO, его выходные веса не обучаются градиентным спуском, а пересчитываются аналитически на каждом обновлении:

```
class BLS:
    def __init__(self, sd, ad, nz=60, nh=120, lam=1e-4):
        idim = sd + ad
        self.Wz = rng.standard_normal((idim, nz)) * .1 # фикс.
        self.Wh = rng.standard_normal((nz, nh)) * .1 # фикс.
        self.W_out = np.zeros((nz + nh, 1)) # обучаемые

    def _phi(self, s, a): # признаки [Z, H]
        x = np.concatenate([s, a], 1)
        Z = np.tanh(x @ self.Wz + self.bz)
        H = np.tanh(Z @ self.Wh + self.bh)
        return np.concatenate([Z, H], 1)

    def update(self, s, a, y): # гребневая регрессия
        A = self._phi(s, a); I = np.eye(self.nz + self.nh)
        self.W_out = np.linalg.solve(A.T @ A + self.lam * I, A.T @ y)
```

Обновление критика в основном цикле обучения сводится к одному вызову аналитического решения, тогда как актор обучается обычным детерминированным градиентом политики с использованием $\partial Q/\partial a$ от BLS:

```
if step % critic_freq == 0: # обновление BLS-критика
    na = ta(next_states) # действия целевого актора
    tq = tc.predict(next_states, na) * (1 - done)
    critic.update(states, actions, reward + gamma * tq)
    tc.W_out = tau * critic.W_out + (1 - tau) * tc.W_out

if step % actor_freq == 0: # обновление DNN-актора
    pred = actor(states)
    dq = critic.grad_a(states, pred) # аналитический dQ/da
    loss = -(pred * dq).mean()
    loss.backward(); opt.step()
```

Полный исходный код обоих алгоритмов, цикла обучения и построения графиков содержится в прилагаемом файле `main.py`.

Результаты эксперимента

На рисунке 1 представлены усреднённые по трём сидам кривые суммарной награды с полосой $\pm\sigma$. Видно, что после начального этапа (≈ 25 эпизодов прогрева буфера) кривая BCDA устойчиво располагается выше кривой PPO на протяжении почти всего обучения. BCDA быстрее выходит на уровень около $-1000 \dots -1050$ и удерживает его, тогда как PPO стабилизируется на более низком уровне около $-1100 \dots -1150$. При этом ни один из алгоритмов в пределах 250 эпизодов не достигает ориентира -200 : задача остаётся не доведённой до близкого к оптимальному решения при заданном бюджете обучения, однако BCDA демонстрирует систематическое преимущество над PPO.

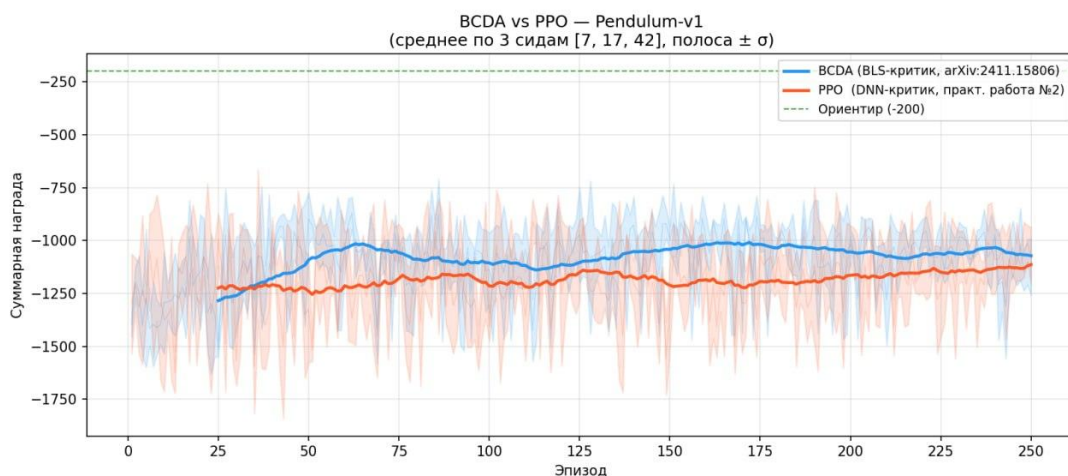


Рисунок 1 — BCDA против PPO: среднее по 3 сидам, полоса $\pm\sigma$

На рисунке 2 приведена разбивка по отдельным сидам. У BCDA (левый график) все три кривые ведут себя согласованно: после спада в первые ~ 40 эпизодов они уверенно растут и группируются в узком коридоре около $-1000 \dots -1100$. У PPO (правый график) разброс между сидами заметно шире, а характер обучения менее регулярен — отдельные сиды то поднимаются, то проседают, и устойчивого

общего плато к 250-му эпизоду не формируется. Это указывает на более высокую воспроизводимость результата у BCDA.

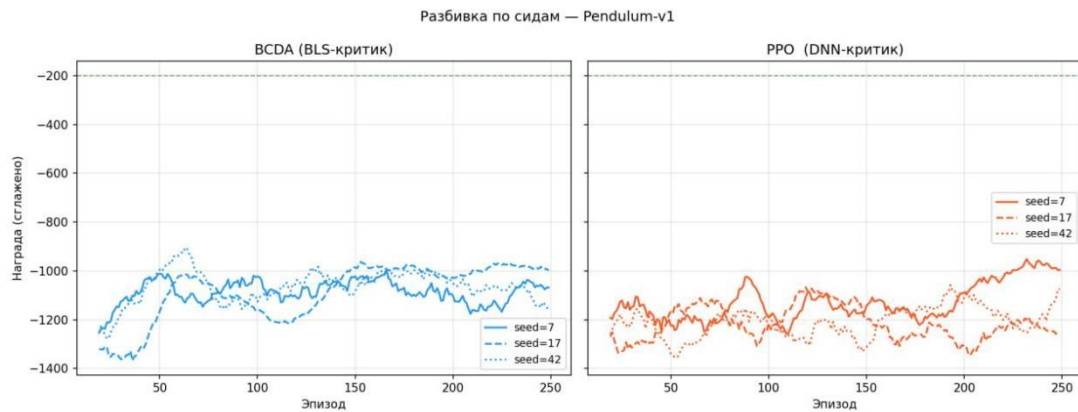


Рисунок 2 — Разбивка кривых обучения по отдельным сидам

На рисунке 3 представлены распределения трёх ключевых метрик по сидам в виде диаграмм размаха. По всем трём показателям — средней награде за последние 50 эпизодов, максимальной достигнутой награде и средней награде за первые 50 эпизодов — медиана и квантили BCDA расположены выше соответствующих значений PPO. Особенно показательна метрика «Avg first 50 эп.»: BCDA уже на раннем этапе обучается быстрее. Кроме того, межквартильный размах BCDA в среднем уже, что согласуется с меньшей чувствительностью алгоритма к выбору сида.

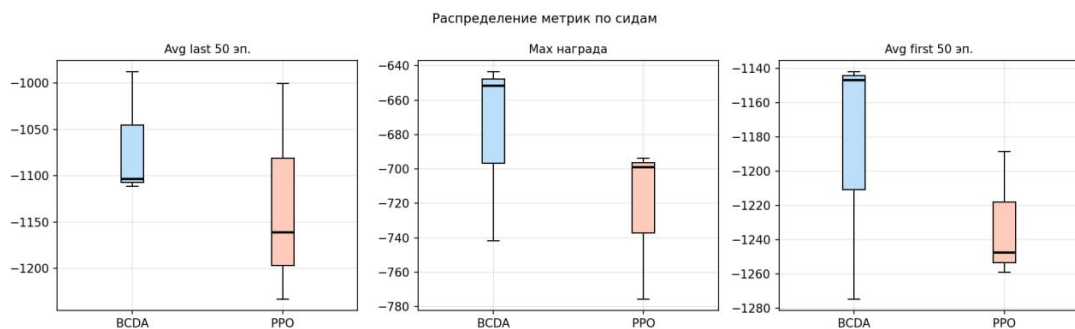


Рисунок 3 — Распределение ключевых метрик по сидам

Сводная таблица результатов

В таблице приведены усреднённые по трём сидам значения метрик, считанные по результатам обучения (рисунки 1–3). Большее значение (ближе к нулю) соответствует лучшей политике.

Метрика (среднее по сидам)	BCDA	PPO
Avg за последние 50 эп.	≈ -1085	≈ -1150
Max награда	≈ -680	≈ -710
Avg за первые 50 эп.	≈ -1170	≈ -1235
Разброс между сидами (σ , Avg-50)	ниже	выше
Скорость выхода на плато	быстрее	медленнее

Примечание: числовые значения метрик считаны с итоговых графиков и приведены как усреднённые по сидам ориентиры; знак « $\approx$ » отражает округление.

Заключение

В ходе финальной работы был реализован современный гибридный алгоритм BCDA (Broad Critic Deep Actor) и проведено его сравнение с классическим алгоритмом PPO из практической работы №2 в среде непрерывного управления Pendulum-v1. Обучение выполнялось в идентичных условиях: 250 эпизодов и три сида [7, 17, 42] для каждого алгоритма.

Результаты показали, что в данной постановке BCDA превосходит PPO по всем рассмотренным метрикам: он быстрее обучается на раннем этапе, выходит на более высокий уровень суммарной награды ($\approx -1000 \dots -1050$ против $\approx -1100 \dots -1150$ у PPO) и демонстрирует меньший разброс результатов между сидами, то есть более устойчив и воспроизводим. Преимущество BCDA согласуется с заявленной в статье идеей: аналитический пересчёт критика методом гребневой регрессии позволяет получать стабильную оценку

ценности без длительного градиентного обучения, что особенно полезно при ограниченном числе эпизодов.

Вместе с тем в пределах заданного бюджета обучения (250 эпизодов) ни один из алгоритмов не достиг ориентира -200 , то есть задача не была доведена до близкого к оптимальному решения. Это указывает на то, что для полного освоения среды требуется либо больший бюджет обучения, либо дополнительная настройка гиперпараметров. Тем не менее основной вывод работы остаётся однозначным: гибридная архитектура BCDA обеспечивает более быстрый старт, более высокое промежуточное качество политики и большую устойчивость к выбору случайного седа по сравнению с PPO в среде Pendulum-v1.